# LoL Champions Visual Data: Reconocer al Campeón a partir de su Splash Art

Emilio López
A01711977@tec.mx

**ABSTRACT** Este reporte muestra cómo construimos una red neuronal convolucional que reconoce a qué campeón de League of Legends pertenece una *splash art*, usando el dataset *"League of Legends Champions Visual Data"*. El reto principal es que cada campeón tiene muy pocas imágenes y que entre sus skins cambian los colores, la ropa y el fondo; lo único que se mantiene es el propio campeón. Durante la limpieza encontramos y quitamos 174 splash arts grupales (la misma imagen etiquetada como campeones distintos) y 18 duplicados, y después de una primera iteración fallida con 107 campeones nos quedamos con los 38 que tienen al menos 15 imágenes limpias. Seguimos un proceso incremental de nueve versiones: una CNN desde cero (v1) se quedó en 0.105 de accuracy en validación, el aumento de datos con Dropout no la mejoró, y el transfer learning con EfficientNetB0, congelada y luego con fine-tuning, llegó a 0.200. El cambio que más ayudó no fue de arquitectura sino de cómo se le muestra la imagen al modelo: dividir la splash en tres recortes de 384 px subió validación a 0.242 y el top-5 a 0.600. Después descubrimos que el color de las skins confunde al modelo: entrenar y predecir en escala de grises dio la mejor versión en validación (accuracy 0.316, F1 macro 0.284). El modelo final, esa misma versión reentrenada con train + validación, llegó a 0.347 de accuracy, 0.558 de top-5 y 0.341 de F1 macro en test, contra 0.026 que daría el azar con 38 clases.

## 1. Introducción

League of Legends tiene más de 170 campeones, y cada uno tiene varias *skins*: versiones alternativas del personaje con otro diseño, otra ropa y otros colores. Cada skin viene con una *splash art*, una ilustración grande del campeón en una escena. Reconocer a un campeón en cualquiera de sus skins es fácil para alguien que juega, porque se fija en la silueta, el arma o las proporciones del personaje, pero para un modelo es bastante más difícil: dos skins del mismo campeón pueden no compartir ni un solo color, y dos campeones distintos pueden tener skins de la misma temática y verse casi iguales.

El objetivo de este trabajo es construir, con TensorFlow y Keras, un clasificador de imágenes que reciba una splash art y diga a qué campeón pertenece, junto con su top 5 de campeones más probables. La idea no es solo llegar a un modelo que funcione, sino documentar todo el proceso: cómo limpiamos los datos, por qué tomamos cada decisión, qué probamos y no funcionó, y qué nos enseñaron los errores del modelo sobre el problema.

Para eso seguimos un proceso incremental: empezamos con una CNN sencilla como aproximación inicial, y fuimos agregando una mejora a la vez (aumento de datos, transfer learning, fine-tuning, cambios en la entrada y reentrenamiento con todos los datos), midiendo el efecto de cada una en el conjunto de validación.

## 2. Descripción del Dataset

El dataset es *League of Legends Champions Visual Data*, de Kaggle [1]. Tiene una carpeta por campeón (172 en total) con 5,872 archivos, de dos tipos:

- **Íconos de habilidades** (archivos `00`–`04`): 860 imágenes PNG de $64 \times 64$.
- **Splash arts** (archivos `05` en adelante): 2,076 imágenes JPG de $1215 \times 717$, una por skin. El archivo `05_<CAMPEÓN>` es siempre la skin original.

Además, cada imagen viene con una copia en escala de grises que termina en `_bw`, así que el total de archivos es el doble.

El número de splash arts por campeón es muy desigual (Figura 1): la media es de 12 por campeón, pero los campeones más recientes tienen solo 2 o 3 (Ambessa, Yunara, Hwei, Mel, Aurora) mientras que los más viejos llegan a 23. Esto ya nos adelantaba que la cantidad de datos iba a ser el problema principal.

## 3. Extracción y Limpieza de Datos

### *3.1.- Íconos contra Splash Arts*

Lo primero que decidimos fue **usar solo las splash arts**. Los íconos de habilidades no muestran al campeón, son efectos, armas o símbolos de $64 \times 64$, así que en realidad son otro problema visual. Mezclarlos

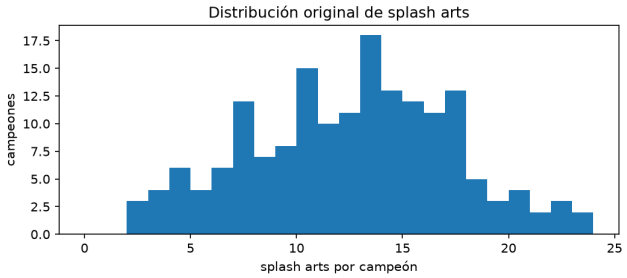



Figura 1: Distribución original de splash arts por campeón.

con las splash obligaría al modelo a aprender dos tareas distintas con muy pocos ejemplos de cada una.

### *3.2.- Splash Arts Grupales y Duplicados*

Al revisar las imágenes encontramos que algunas skins son **grupales**: la misma imagen muestra a varios campeones (por ejemplo, las skins de equipos de esports como DRX o SKT T1), y en el dataset esa imagen aparece copiada en la carpeta de cada uno (Figura 2). Para el modelo eso es la misma entrada con etiquetas distintas, algo imposible de aprender que además ensucia la evaluación.

También había imágenes repetidas dentro de un mismo campeón (por ejemplo, `PRESTIGE_KDA_AHRI` y `PRESTIGE_KDA_AHRI_2022`). Si una cae en train y la otra en test, el resultado se infla, porque el modelo estaría "reconociendo" una imagen que ya vio.

Para detectar ambos casos usamos dos huellas por imagen:

- **MD5**: detecta archivos idénticos byte por byte.
- **Average hash**: reduce la imagen a $16 \times 16$ en grises y marca qué pixeles están por encima de la media. Detecta imágenes casi iguales aunque tengan otra compresión o un recorte un poco distinto.

Si una huella se repetía entre campeones distintos la marcamos como grupal, y si se repetía dentro del mismo campeón, como duplicado. En total quitamos **174 splash grupales** y **18 duplicados**. Los campeones más afectados fueron Rakan y Xayah (7 grupales cada uno), que comparten casi todas sus skins.

Este método tiene una limitación: solo encuentra una grupal cuando la misma imagen está en más de una carpeta. En la revisión visual vimos que las splash restantes con más de un personaje muestran al campeón principal ocupando la mayor parte de la imagen, y a los demás como secundarios. Esas sí las conservamos: lo que se repite entre las skins de un campeón es el propio campeón, mientras que los secundarios cambian de una skin a otra, así que solo aportan un poco de ruido.

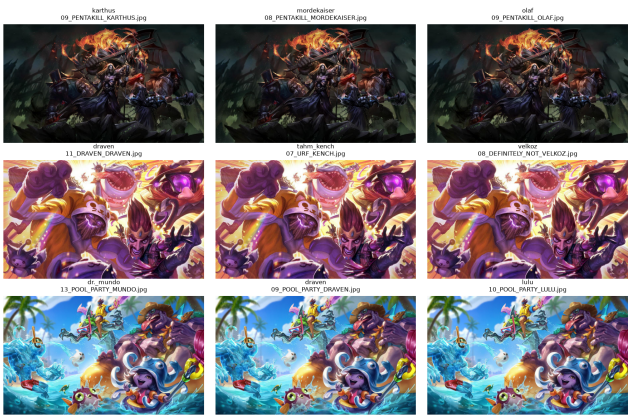


Figura 2: Ejemplos de splash arts grupales: la misma imagen aparece en la carpeta de cada campeón.

### *3.3.- Filtro de Campeones*

Después de quitar grupales y duplicados, varios campeones se quedaban con 2 o 3 imágenes, y con tan pocas no se puede tener al menos un ejemplo en cada split y además entrenar. Decidimos conservar solo los campeones con al menos `MIN_SPLASH` imágenes limpias. La Tabla 1 muestra cuántos campeones quedan con cada umbral.

| Mínimo | Campeones | Imágenes |
|---|---|---|
| 5 | 159 | 1,842 |
| 8 | 126 | 1,635 |
| 10 | 107 | 1,470 |
| 12 | 77 | 1,152 |
| 15 | 38 | 648 |

Tabla 1: Campeones e imágenes que quedan según el mínimo de splash arts limpias.

**Primero probamos con `MIN_SPLASH = 10`** (107 campeones, ∼9 imágenes por campeón en train), porque queríamos que el clasificador reconociera a la mayor cantidad posible de campeones. No funcionó: ningún modelo superó ∼10% de accuracy en validación. Para saber si el problema era la arquitectura o los datos, hicimos un diagnóstico con cuatro redes preentrenadas como extractores de características, entrenando solo un clasificador encima de cada una (Tabla 2).

Las cuatro redes llegaron prácticamente al mismo techo, así que el límite no estaba en la arquitectura sino en los datos: con ∼9 skins por campeón, que además no se parecen entre sí, no hay suficientes ejemplos para que el modelo encuentre qué tienen en común. Al repetir la prueba con EfficientNetB0 y `MIN_SPLASH = 15`, la accuracy en validación subió de 0.116 a 0.263. Por eso nos quedamos con **38 campeones y 648 splash arts**. Reconocer menos campeones pero hacerlo bien nos pareció

| Red (107 campeones) | Val acc | Val top-5 |
|---|---|---|
| MobileNetV2 | 0.099 | 0.249 |
| ResNet50 | 0.107 | 0.245 |
| EfficientNetB0 | **0.116** | **0.262** |
| EfficientNetB3 | 0.107 | 0.249 |
| EfficientNetB0, 38 campeones | 0.263 | 0.547 |

Tabla 2: Diagnóstico con redes preentrenadas congeladas en la primera iteración.

mejor que reconocer muchos al nivel del azar.

### *3.4.- División del Dataset*

Dividimos los datos en 70% train, 15% validación y 15% test, de forma **estratificada por campeón** (cada campeón aparece en los tres conjuntos, con mínimo una imagen en val y una en test) y con semilla fija para que sea reproducible.

Tomamos dos decisiones importantes en esta parte:

- **La splash base siempre va a train.** La skin original es la apariencia oficial del campeón y la más parecida a cómo se ve en el juego, así que es el ejemplo más representativo de cada clase. Con tan pocas imágenes no nos convenía quitárselo al entrenamiento. Así, val y test tienen solo skins que el modelo nunca vio, y miden justo lo que queremos: si reconoce al campeón aunque cambie todo lo demás.
- **Las copias `_bw` solo se usan en train.** Son la misma skin en escala de grises, así que meterlas en val o test sería evaluar dos veces la misma imagen. En train duplican los ejemplos y obligan al modelo a fijarse en algo más que el color.

El resultado fue 458 splash en train (916 con sus copias `_bw`), 95 en validación y 95 en test, entre 11 y 15 por campeón en train y 2 o 3 en val y test (Figura 3). Al final verificamos con las dos huellas que ninguna imagen, ni casi-duplicado, aparece en más de un split.

Algo que hay que tener en cuenta en todo el reporte: **val y test tienen 95 imágenes cada uno**, así que cada imagen vale ∼0.011 de accuracy. Diferencias de una o dos imágenes entre versiones están dentro del ruido.

## 4. Transformación de Datos

### *4.1.- De Imagen Ancha a Cuadrada*

Las splash miden $1215 \times 717$ (proporción ∼1.7:1), pero las redes preentrenadas de Keras esperan entradas cuadradas, normalmente de $224 \times 224$. Comparamos tres formas de convertirlas (Figura 4):

- **Estirar**: usa toda la imagen, pero deforma al campeón horizontalmente.
- **Recortar el centro**: no deforma, pero pierde ∼40% del ancho, y en muchas splash el campeón no está centrado.
- **Letterbox** (rellenar con bordes negros): no deforma ni pierde nada, pero ∼40% de la entrada son bordes y el campeón queda todavía más pequeño.

En la segunda iteración probamos las tres con el mismo modelo. Estirar (F1 macro de validación 0.121) y recortar el centro (0.106) quedaron empatados dentro del ruido, y letterbox fue claramente peor (0.068). Nos quedamos con estirar para las versiones de 224 px, pero la lección más importante fue otra: **el tamaño con el que se ve al campeón importa más que la forma de pasar a cuadrado**, y eso es lo que llevó a la v5 (sección 5.7).

### *4.2.- Reducción de Tamaño*

Guardar las splash a $1215 \times 717$ solo hacía más lenta la carga, así que las guardamos conservando la proporción con 384 px de alto ($\sim 651 \times 384$), que es la resolución más grande que usa cualquier versión. Usamos el filtro LANCZOS, que reduce sin generar artefactos. Las versiones de 224 px reducen la imagen al cargarla.

### *4.3.- Normalización*

La CNN desde cero recibe los pixeles normalizados a $[0, 1]$ con una capa `Rescaling(1/255)`. EfficientNet trae su propia normalización incluida en el modelo, así que a esas versiones les pasamos los pixeles en $[0, 255]$ directamente.

## 5. Construcción del Modelo

### *5.1.- Métricas y Criterio de Selección*

Reportamos tres métricas para cada versión:

- **Accuracy**: porcentaje de aciertos.
- **Top-5 accuracy**: si el campeón correcto está entre las 5 predicciones más probables. Con 38 clases es una medida útil de qué tan "cerca" está el modelo aunque falle.
- **F1 macro**: el promedio del F1 de cada campeón, todos con el mismo peso. Un modelo no puede verse bien acertando solo a unos cuantos campeones.

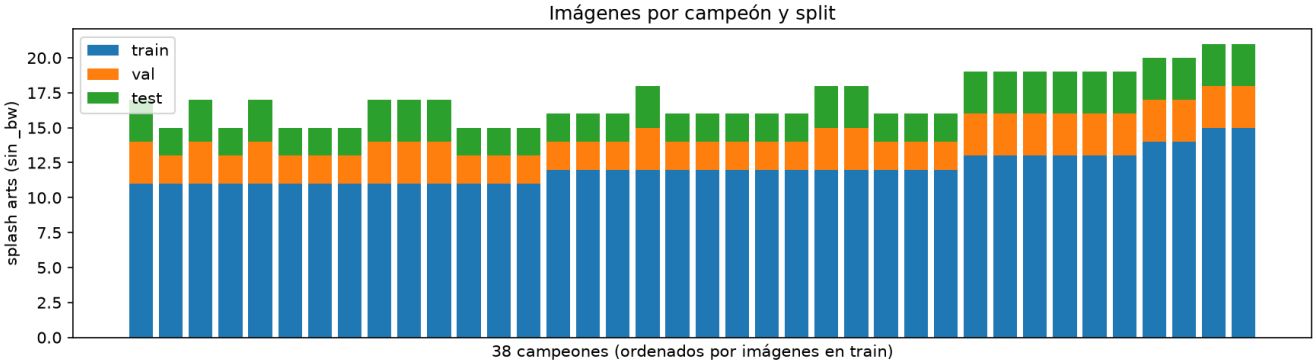



Figura 3: Imágenes por campeón en cada split. El desbalance entre clases es pequeño (11 a 15 en train).

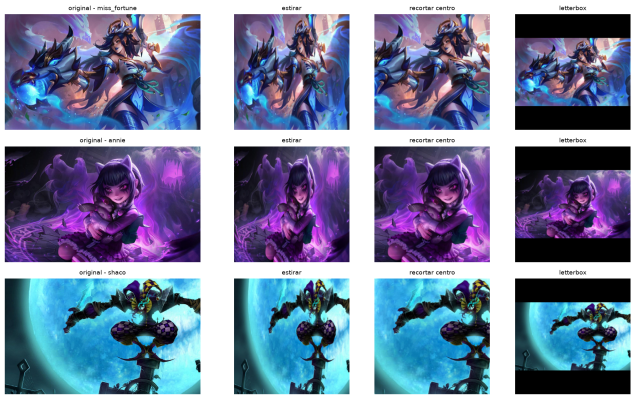



Figura 4: Tres formas de convertir una splash ancha en una entrada cuadrada.

**Todas las decisiones se tomaron con validación**, usando el F1 macro como criterio. Test se reporta para cada versión pero nunca se usó para elegir nada.

En la primera iteración usábamos `EarlyStopping` sobre `val_loss`, y eso cortaba el entrenamiento demasiado pronto: la pérdida de validación sube en cuanto el modelo se vuelve demasiado confiado en sus errores, aunque el número de aciertos siga mejorando. En la v1 de esa iteración el modelo memorizó el 100% de train en 3 épocas y se restauraron los pesos de la época 0. Por eso cambiamos a vigilar el F1 macro.

### *5.2.- Entrenamiento*

Todas las versiones se entrenan con la misma función, para que la comparación sea justa:

- **Salida**: capa densa de 38 neuronas con softmax.
- **Pérdida**: *sparse categorical crossentropy*, porque las etiquetas son enteros:

$$J = -\frac{1}{m}\sum_{i=1}^{m} \log \hat{p}_{i,y_i} \qquad (1)$$

donde $\hat{p}_{i,y_i}$ es la probabilidad que el modelo le da al campeón correcto de la imagen $i$.

- **Optimizador**: Adam, con batches de 32.
- **Callback propio `ValF1`**: al final de cada época predice validación y calcula el F1 macro, para que los demás callbacks lo puedan usar.
- **`EarlyStopping`** sobre el F1 macro de validación: para cuando deja de mejorar y restaura los pesos de la mejor época.
- **`ReduceLROnPlateau`** sobre el mismo F1: si se estanca, divide el learning rate entre 2, para dar pasos más pequeños cuando el optimizador ya no avanza.

Cada versión se guarda en formato `.keras` (arquitectura y pesos) tal como se evaluó.

### *5.3.- v1: CNN desde Cero*

Como aproximación inicial armamos una CNN sencilla, con la misma estructura que el ejemplo de perros y gatos que vimos en clase: tres bloques de convolución $3 \times 3$ con ReLU (32, 64 y 128 filtros), cada uno seguido de un *max pooling* que reduce la resolución a la mitad, y al final un `Flatten`, una capa densa de 256 neuronas y la salida softmax. Tiene 22.3 millones de parámetros, casi todos en la capa densa (el `Flatten` genera 86,528 entradas).

La v1 llegó al **100% de accuracy en train** pero solo a 0.105 en validación (Figura 5): con 916 imágenes y 22 millones de parámetros, la red simplemente memorizó el conjunto de entrenamiento. El resultado está por encima del azar (0.026), pero muy lejos de ser útil.

### *5.4.- v2: Aumento de Datos y Dropout*

Para atacar el sobreajuste agregamos dos técnicas de regularización:

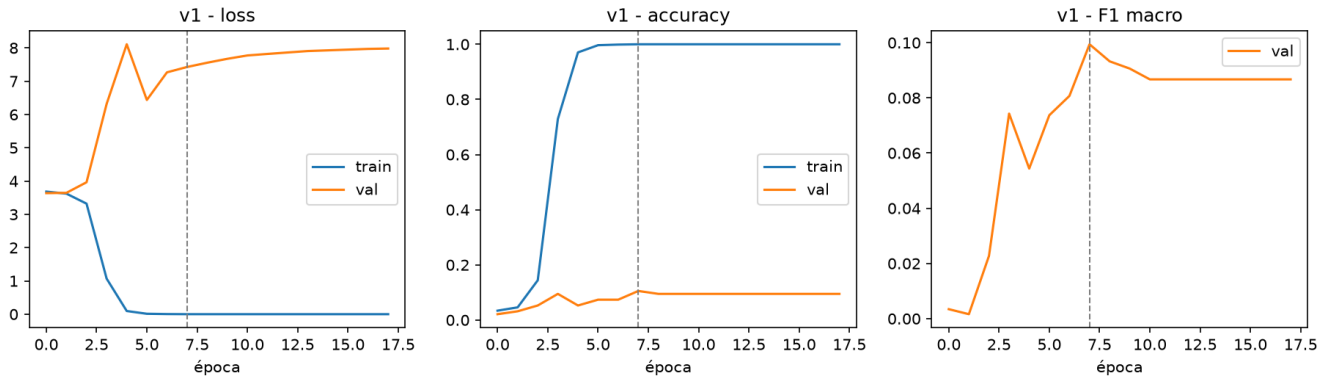



Figura 5: Curvas de entrenamiento de la v1. Train llega al 100% mientras la pérdida de validación se dispara: el modelo memoriza. La línea punteada marca la mejor época según el F1 macro de validación.

- **Aumento de datos**: en cada época cada imagen llega con una transformación aleatoria distinta (volteo horizontal, rotación de ±5%, zoom de ±15%, desplazamiento de ±10% y cambios de contraste), así el modelo no la puede memorizar (Figura 6). No usamos volteo vertical porque un campeón de cabeza no es una imagen realista. Estas capas solo actúan durante el entrenamiento.
- **Dropout** de 0.5 antes y después de la capa densa: apaga al azar la mitad de las neuronas en cada paso, para que el clasificador no dependa de unas pocas características.

El resultado fue peor: **0.053 en validación y 0.095 en train** (Figura 7). Ya no memoriza, pero ahora tampoco aprende: con tan pocas imágenes, la red desde cero no tiene de dónde sacar características generales (bordes, texturas, formas) y la regularización solo le quitó la capacidad de memorizar. Esto nos dejó claro que el problema no se resolvía regularizando una red desde cero, sino dándole al modelo características que ya sepan "ver".

### *5.5.- v3: Transfer Learning con EfficientNetB0*

El **transfer learning** reutiliza una red ya entrenada con ImageNet (1.4 millones de imágenes), que ya sabe extraer bordes, texturas y formas, y solo entrena un clasificador nuevo encima. Elegimos **EfficientNetB0** [2] porque en el diagnóstico de la Tabla 2 fue la de mejores características congeladas, y porque es una red ligera que corre bien en CPU. EfficientNet escala de forma balanceada la profundidad, el ancho y la resolución, y usa bloques MBConv con convoluciones *depthwise* y atención por canal.

- `include_top=False` quita el clasificador original de 1000 clases de ImageNet.
- `base.trainable = False` congela los 4 millones de parámetros de la base. Solo se entrenan 48,678.
- `GlobalAveragePooling2D` resume cada mapa de características en un número, en lugar del `Flatten` de la v1, que generaba 22 millones de parámetros.
- Mantenemos el aumento de datos y un Dropout de 0.3 antes de la salida.

La v3 subió a **0.179 en validación** (F1 0.165) y a 0.368 en top-5, con un sobreajuste mucho menor que la v1 (0.639 en train).

### *5.6.- v4: Fine-Tuning*

Con el clasificador de la v3 ya entrenado, **descongelamos el último bloque** de EfficientNetB0 para que también se ajustara a las splash arts. Las primeras capas detectan cosas genéricas que sirven para cualquier imagen, mientras que las últimas detectan patrones más específicos de ImageNet, que son las que conviene adaptar.

- Descongelamos desde `block6a` (bloques 6 y 7 y la convolución final): 59 de 238 capas y 3.2 millones de parámetros. Descongelar más eran más parámetros que reentrenar con pocos datos y más riesgo de perder lo aprendido en ImageNet.
- Usamos un learning rate 100 veces menor ($10^{-5}$) para hacer ajustes pequeños sin destruir los pesos preentrenados.
- Las capas de *Batch Normalization* se quedaron congeladas, porque sus estadísticas se calcularon con millones de imágenes y recalcularlas con batches de 32 las desestabiliza.
- Partimos de los pesos de la v3: si el clasificador empezara aleatorio, sus gradientes dañarían la base en las primeras épocas.

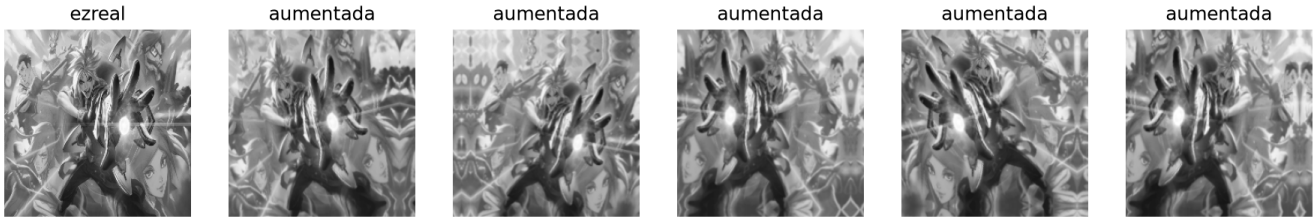



Figura 6: Una imagen de train (en este caso una copia `_bw`) y cinco versiones aumentadas al azar.

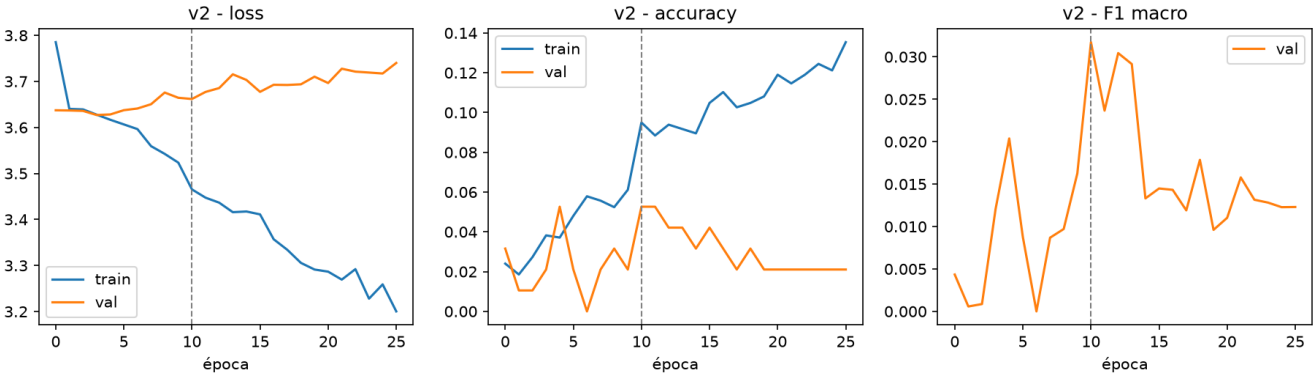



Figura 7: Curvas de entrenamiento de la v2. Con aumento de datos y Dropout, la accuracy de train no pasa de 0.14: el modelo deja de memorizar, pero tampoco aprende.

La v4 mejoró un poco en validación (**0.200**, F1 0.173), pero test no se movió (0.126 en ambas). En la segunda iteración ya habíamos visto lo mismo: cambiar de backbone, la forma de pasar a cuadrado o el aumento de datos no movía test de ∼0.17. Algo más estaba limitando al modelo.

### *5.7.- v5: Multi-Recorte a 384 px*

Revisando las predicciones de la iteración anterior notamos un patrón: el modelo fallaba justo cuando el campeón era pequeño dentro de una escena cargada, y en esos casos respondía por la paleta de colores de la imagen. Al aplastar una splash de 1.7:1 a $224 \times 224$, el campeón queda pequeño y deformado.

La idea de la v5 fue **no aplastar la imagen**: dividimos la splash de $651 \times 384$ en **tres recortes cuadrados de** $384 \times 384$ (izquierda, centro y derecha, que se traslapan), pasamos cada uno por la misma EfficientNetB0 y **concatenamos** los tres vectores de características (Figura 8). Así el campeón se ve sin deformar y con ∼1.7 veces más resolución, y el clasificador sabe en qué zona de la imagen apareció cada característica. En un diagnóstico previo, concatenar dio mejor F1 en validación (0.274) que promediar los tres recortes (0.226).

En Keras armamos los recortes dentro del propio modelo:

- Tres capas `Cropping2D` toman la misma entrada de $651 \times 384$ y le quitan columnas a la derecha (recorte izquierdo), a ambos lados (centro) o a la izquierda (recorte derecho), dejando cuadrados de $384 \times 384$.
- Los tres recortes pasan por **la misma instancia** de EfficientNetB0, así que comparten pesos: el modelo no tiene más parámetros que con una sola entrada, solo procesa la imagen tres veces.
- Una capa `Concatenate` junta las tres salidas siempre en el mismo orden (izquierda, centro, derecha). Por eso el clasificador sabe de qué zona viene cada característica.

Procesar tres recortes de 384 px por imagen en cada época era muy lento en CPU, así que usamos la base como **extractor de características fijo**, una de las formas de transfer learning que vimos en clase: como está congelada, sus características no cambian entre épocas, así que las calculamos **una sola vez** ($3 \times 1280 = 3840$ por imagen) y después entrenamos solo el clasificador (Dropout de 0.3 y la capa softmax), en segundos por época.

El costo es que no se puede usar aumento de datos aleatorio. En su lugar agregamos una copia **volteada horizontalmente** de cada imagen de train (1,832 ejemplos). En la segunda iteración habíamos visto que quitar el aumento aleatorio no empeoraba los resultados, así que no perdíamos mucho.




Figura 8: Una splash completa y sus tres recortes cuadrados de 384 × 384.

Al final unimos el extractor y el clasificador en un solo modelo que recibe la splash completa. La v5 fue el salto más grande: **0.242 en validación y en test**, y el top-5 de validación pasó de 0.337 a **0.600**.

#### *5.8.- v6: Reentrenar con Train + Validación*

Todas las versiones hasta aquí mostraban el mismo patrón: el modelo llega a ∼99% en train y se queda muy por debajo en validación. Cambiar la arquitectura ayudó, pero el sobreajuste seguía ahí, y nuestra hipótesis era que el límite es la **cantidad de datos**.

En clase vimos que, una vez elegidos el modelo y sus hiperparámetros con validación, se puede reentrenar con todos los datos disponibles, porque validación ya cumplió su función. La v6 es exactamente la v5 (mismo extractor, mismo clasificador, mismo learning rate en cada época y las 29 épocas de su mejor resultado), pero entrenada con train + validación (y el volteo de validación), lo que agrega 95 splash nuevas, ∼2.5 por campeón. Como lo único que cambia es la cantidad de datos, si test mejora es por eso.

Test subió de **0.242 a 0.305** (de 23 a 29 aciertos: 8 imágenes pasaron de error a acierto y 2 al revés), y el F1 de 0.242 a 0.300. Esto es consistente con la hipótesis, aunque la v6 sigue llegando al 99% en train: con más datos generaliza mejor, pero el límite sigue siendo cuántas skins hay por campeón. La v6 ya no tiene métricas de validación, porque la usó para entrenar.

## 6. ¿El Modelo Depende del Color?

Entre las skins de un campeón lo que más cambia es la paleta de colores, mientras que la forma, la silueta y las proporciones se mantienen. Si el clasificador se apoya en el color, estaría aprendiendo pistas que no se repiten de una skin a otra, y eso explicaría los errores en los que respondía por la paleta de la imagen.

#### *6.1.- Diagnóstico*

Para probarlo evaluamos la v5 y la v6, tal como se entrenaron (color más copias `_bw`), con validación y test convertidos a escala de grises. Si el modelo dependiera del color, los resultados deberían bajar.

| Modelo / Split | Color | Grises |
|---|---|---|
| v5 – Validación | 0.242 (23) | **0.284 (27)** |
| v5 – Test | 0.242 (23) | **0.274 (26)** |
| v6 – Test | 0.305 (29) | **0.337 (32)** |

Tabla 3: Accuracy (y aciertos de 95) de la v5 y la v6 evaluadas a color y en escala de grises.

Pasó lo contrario (Tabla 3): en los tres casos el modelo **acierta más en grises que a color**. No depende del color; al contrario, el color de las skins le estorba. Eso nos llevó a entrenar directamente en grises.

#### *6.2.- v7: Entrenar y Predecir en Escala de Grises*

Si el clasificador solo ve imágenes en grises al entrenar, no puede aprender pistas de color y se tiene que apoyar en la forma del campeón. La v7 es la misma v5, cambiando solo los datos: en train usa solo las copias `_bw` y su volteo, y validación y test se convierten a grises antes de predecir.

La v7 fue **la mejor versión en validación**: accuracy 0.316 (30 aciertos contra 23 de la v5) y F1 macro 0.284 (contra 0.212). En test también mejoró a la v5 (0.284 contra 0.242). Lo único que no mejoró fue el top-5 (0.589 contra 0.600): cuando el modelo en grises falla, el campeón correcto queda un poco menos veces entre las 5 primeras opciones.

#### *6.3.- v8: Color y Grises con más Peso a los Grises*

Quitar el color por completo también quita las pistas de color que sí se mantienen entre skins, como el tono de piel de algunos campeones. Para buscar un punto

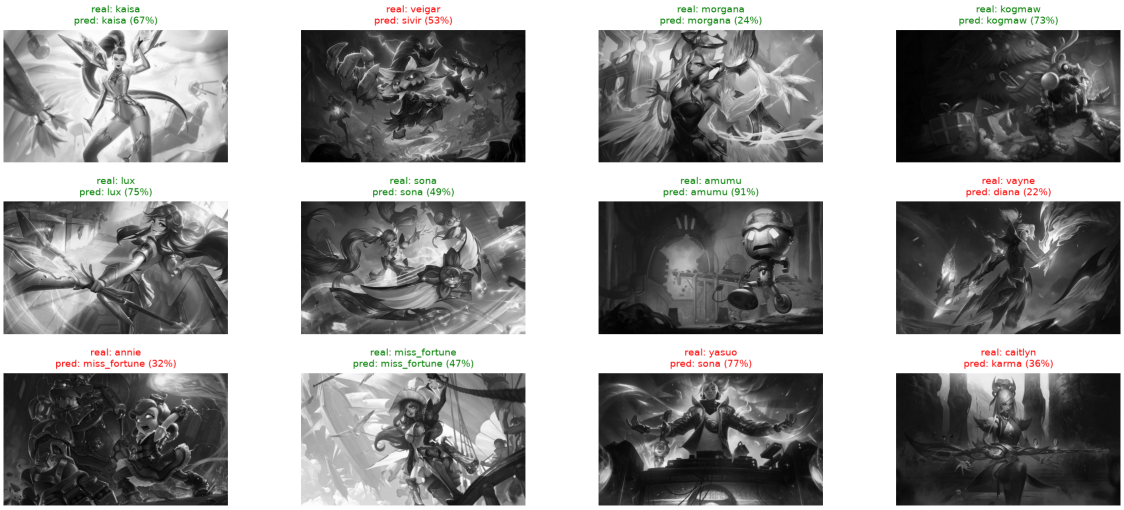



Figura 9: Predicciones del modelo final sobre imágenes de test, ya convertidas a escala de grises. En verde los aciertos y en rojo los errores, con la probabilidad que el modelo le dio a su predicción.

intermedio, la v8 entrena con todas las imágenes de train (a color y `_bw`, más sus volteos), pero cada imagen a color cuenta la mitad en la pérdida, y se evalúa en grises igual que la v7.

Para eso usamos `sample_weight`, que le asigna un peso $w_i$ a cada ejemplo dentro de la pérdida:

$$J = -\frac{1}{m}\sum_{i=1}^{m} w_i \log \hat{p}_{i,y_i} \tag{2}$$

Las copias `_bw` tienen $w_i = 1$ y las imágenes a color (con sus volteos) $w_i = 0.5$. Así, equivocarse en una imagen a color mueve los pesos la mitad que equivocarse en una en grises: el modelo sigue viendo el color, pero lo que más le conviene aprender son las características que también funcionan en grises. Con $w_i = 1$ para todas, el entrenamiento sería el de la v5, y con $w_i = 0$ para las de color, prácticamente el de la v7, así que la v8 queda entre las dos. En Keras basta con que el dataset regrese `(x, y, w)` en lugar de `(x, y)`.

Fijamos el peso en 0.5 antes de ver resultados: probar varios pesos y quedarnos con el mejor hubiera sido ajustar a validación con solo 95 imágenes.

La v8 quedó prácticamente empatada con la v7: 29 aciertos contra 30 en validación y F1 de 0.280 contra 0.284. Recuperó algo de top-5 (0.611 contra 0.589), lo que confirma que el color sí aporta como pista secundaria. Pero las diferencias son de 1–2 imágenes, dentro del ruido, y con nuestra regla (elegir por F1 macro en validación) gana la v7, que además es más simple: no necesita las imágenes a color ni un peso que ajustar.

### *6.4.- v9: Modelo Final*

Igual que la v6 con la v5, la v9 es exactamente la v7 reentrenada con train + validación en grises (31 épocas, las de su mejor resultado). Con eso test pasó de 27 a 33 aciertos: **0.347 de accuracy y 0.341 de F1 macro**. Frente a la v6, el mejor modelo a color, la v9 acierta 4 imágenes más y tiene mejor F1, aunque menor top-5 (0.558 contra 0.600).

## 7. Visualización y Resultados

La Tabla 4 resume todas las versiones y la Figura 10 compara las que tienen métricas de validación.

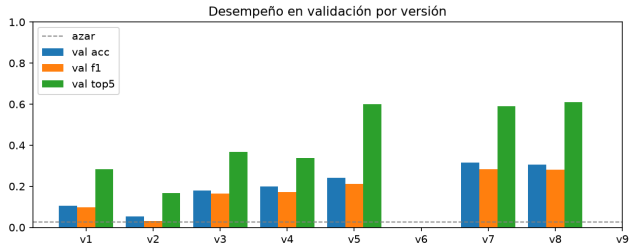



Figura 10: Accuracy, F1 macro y top-5 en validación por versión. La línea punteada es el azar.

Viendo la tabla completa, hay tres cosas que destacan:

- **El tamaño con el que se ve al campeón importó más que la arquitectura.** Entre v3 y v4 (y en las iteraciones anteriores, cambiando de backbone) validación se movió poco y test nada; dividir la splash en tres recortes a mayor resolución (v5) fue el salto más grande.

| Versión | Cambio | Train acc | Val acc | Val F1 | Test acc | Test top-5 | Test F1 |
|---|---|---|---|---|---|---|---|
| v1 | CNN desde cero (224 × 224) | 1.000 | 0.105 | 0.099 | 0.105 | 0.305 | 0.105 |
| v2 | v1 + aumento de datos + Dropout | 0.095 | 0.053 | 0.032 | 0.063 | 0.189 | 0.036 |
| v3 | EfficientNetB0 congelada + aumento | 0.639 | 0.179 | 0.165 | 0.126 | 0.421 | 0.108 |
| v4 | v3 + fine-tuning desde `block6a` | 0.664 | 0.200 | 0.173 | 0.126 | 0.411 | 0.098 |
| v5 | 3 recortes de 384 px, EfficientNetB0 como extractor | 0.997 | 0.242 | 0.212 | 0.242 | 0.516 | 0.242 |
| v6 | v5 reentrenada con train + val | 0.995 | – | – | 0.305 | **0.600** | 0.300 |
| v7 | v5 entrenada y evaluada en grises | 0.995 | **0.316** | **0.284** | 0.284 | 0.537 | 0.263 |
| v8 | v5 color (peso 0.5) + grises, evaluada en grises | 0.997 | 0.305 | 0.280 | 0.295 | 0.537 | 0.277 |
| **v9** | **v7 reentrenada con train + val (final)** | 0.992 | – | – | **0.347** | 0.558 | **0.341** |

Tabla 4: Resultados de todas las versiones. El azar con 38 clases es 0.026. La v6 y la v9 se entrenaron con validación, así que solo tienen métricas de test.

- **El color de las skins confunde al modelo.** Evaluar en grises mejora a un modelo entrenado a color, y entrenar solo en grises da la mejor versión en validación.
- **El límite son los datos.** Todas las versiones con transfer learning a 384 px llegan a ∼99% en train, y agregar las 95 imágenes de validación mejoró test tanto a color (v5 → v6) como en grises (v7 → v9), en ambos casos con 8 imágenes que pasaron de error a acierto y 2 al revés.

### *7.1.- Análisis del Modelo Final en Test*

El modelo final (v9) acierta 33 de las 95 imágenes de test, y en el 55.8% de los casos el campeón correcto está en su top 5. El desempeño por campeón es muy desigual: Kai'Sa, Annie, Blitzcrank y Teemo llegan a un F1 de 0.80, mientras que **13 de los 38 campeones tienen F1 de 0** (entre ellos Kayle, Veigar, Shaco, Riven, Katarina, Fiora y Ezreal). Hay que tomar esto con cuidado, porque cada campeón tiene solo 2 o 3 imágenes en test.

La matriz de confusión (Figura 11) muestra que los errores no se reparten al azar: el modelo tiene campeones "comodín" a los que manda muchas imágenes que no reconoce, como Master Yi (6 errores), Sivir (5), Fiora, Vayne y Jax (4 cada uno). Son campeones humanoides con arma, una silueta que se parece a la de muchos otros.

La Figura 9 muestra predicciones sobre imágenes de test. El modelo acierta con seguridad a campeones de silueta única, como Amumu (91%) o Kog'Maw (73%), y falla con humanoides parecidos entre sí: Vayne la confunde con Diana, Yasuo con Sona y Caitlyn con Karma. También se ve un caso de splash con varios personajes (Annie con un personaje grande al lado), donde el modelo responde por otro campeón.

## 8. Interfaz Web

Para mostrar cómo funciona el modelo armamos una interfaz web sencilla. El modelo final se guarda en formato `.keras` junto con un archivo `metadata.json` que tiene la lista de campeones, el tamaño de entrada y si el modelo espera la imagen en escala de grises. Un servidor en Python (`src/app.py`) carga el modelo una sola vez, recibe las imágenes que sube el usuario, les aplica el mismo preprocesamiento del entrenamiento y regresa el top 5 de campeones con su probabilidad.

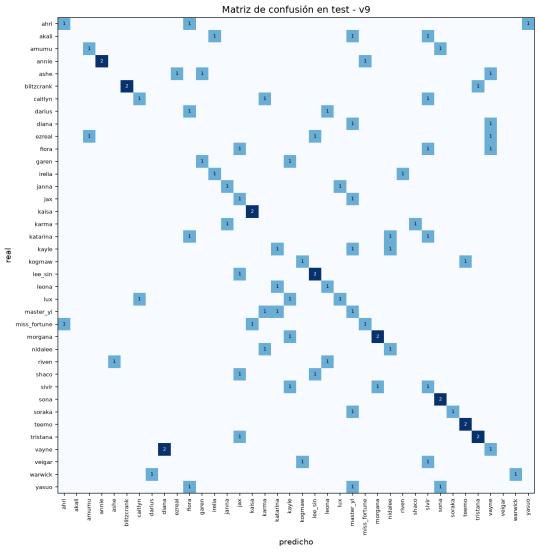



Figura 11: Matriz de confusión del modelo final en test.

Como el modelo se entrenó con splash horizontales, las imágenes con una proporción muy distinta (verticales o cuadradas) no se pueden estirar a 651 × 384 sin deformarlas mucho. Si la proporción se aleja más de 15% de la de una splash, la imagen se coloca completa con bordes negros. Las imágenes que usamos para probar la interfaz no son parte de la evaluación: la evaluación del modelo es la de test.

## 9. Conclusiones

Construimos un clasificador que reconoce a 38 campeones de League of Legends a partir de sus splash arts con 0.347 de accuracy, 0.558 de top-5 y 0.341 de F1

macro en test, 13 veces más que el azar. Es un resultado moderado, pero con 12 skins por campeón en train que no comparten colores, ropa ni fondo, el problema es bastante más difícil de lo que parece.

Lo más valioso del proceso fueron las decisiones que salieron de entender por qué fallaba el modelo, más que de probar arquitecturas: limpiar las splash grupales que eran imposibles de aprender, reducir los campeones cuando vimos que todas las redes llegaban al mismo techo, mostrarle al campeón a mayor resolución cuando vimos que fallaba con los campeones pequeños, y quitar el color cuando vimos que el color de las skins le estorbaba.

El límite que queda es claro: todas las versiones memorizan train y el modelo mejora cada vez que le damos más imágenes. La mejora con más impacto sería conseguir más splash arts por campeón (por ejemplo, de otras fuentes oficiales del juego), lo que además permitiría volver a incluir a los campeones que quitamos.

## Referencias


[1] Elimam, H. E. (n.d.). *League of Legends Champions Visual Data*. Kaggle. https://www.kaggle.com/datasets/houssemeddineelimam/league-of-legends-champions-visual-data

[2] Tan, M., & Le, Q. V. (2019). EfficientNet: Rethinking model scaling for convolutional neural networks. *Proceedings of the 36th International Conference on Machine Learning*, 6105–6114.